

Entrega de Banco de Dados 2

Apresenta os requisitos da disciplina, como o projeto os atende e o arquivo que comprova cada um.

Período	Responsável	Versão	Classificação
2026	**Equipe do projeto**	**1.0 · 02/11/2026**	**Entrega acadêmica**

Sumário

Os requisitos da disciplina e as evidências

01
Visão geral 3
O que a disciplina pede e o que foi entregue

02
Requisitos mínimos 4
MongoDB em conversas e Redis como fila

03
Requisitos extras 5
Neo4j com grafo e traversal

04
Repositórios da disciplina 6
Papel de cada repositório na entrega

01

O projeto usa MongoDB, Redis e Neo4j em seus serviços

O que a disciplina pede e o que a equipe entregou?

O que a disciplina pede

Banco de Dados 2 pede o uso de bancos não relacionais em uma aplicação. O mínimo para a nota 7,0 tem dois requisitos, usar o MongoDB em uma interação conversacional e usar o Redis para fila de processamento ou ranking em tempo real. O extra pede o uso do Neo4j com as entidades principais modeladas como grafo.

O que foi entregue

O MongoDB guarda o estado de cada conversa dos agentes de inteligência artificial e também as conversas e mensagens do serviço de mensagens. O Redis sustenta a fila de eventos do serviço de autenticação, com publicação, grupo de consumidores, confirmação de cada evento e reprocessamento das mensagens pendentes. O Neo4j modela fornecedores, ofertas, modelos de equipamento, profissionais, profissões e turnos, e responde às consultas de recomendação por traversal do grafo.

Como ler este documento

Cada requisito aparece com o identificador, o enunciado, a descrição do que foi entregue e um link que abre direto o arquivo de exemplo na branch main do GitHub. Quando o mesmo padrão existe em outros repositórios, o texto cita apenas o nome deles. A tabela a seguir resume os requisitos.

Item	Tipo	Requisito	Arquivo de exemplo
M1	Mínimo	MongoDB em interação conversacional	mongo_checkpointer.py em ai-assistant
M2	Mínimo	Redis para fila de processamento	OutboxStreamPublisher.kt em api-auth
E1	Extra	Neo4j com traversal	candidates.cypher em api-recommendation

02

As conversas ficam no MongoDB e a fila de eventos fica no Redis

Como o projeto cumpre os dois requisitos do mínimo para 7,0?

M1. Usar o MongoDB em alguma interação conversacional

O agente conversacional do projeto guarda o estado de cada conversa no MongoDB. O checkpointer do LangGraph grava o ponto de cada turno, de modo que uma conversa continua de onde parou. O repositório ai-assistant também mantém no MongoDB a memória de longo prazo de cada usuário.

O mesmo mecanismo de checkpointer existe no agente operacional, em ai-operational. No serviço de mensagens, em api-messenger, as conversas entre pessoas e com o chatbot e as mensagens são documentos de coleções do MongoDB. O api-notify também usa o MongoDB para o histórico de notificações.

Link de exemplo, mongo_checkpointer.py em ai-assistant.

M2. Usar o Redis para fila de processamento e/ou ranking em tempo real

O serviço de autenticação usa uma Redis Stream como fila de processamento de eventos de usuário. A classe OutboxStreamPublisher publica cada evento na stream com XADD. O OutboxPollingScheduler segue o padrão outbox e só marca o evento como publicado depois que o XADD é confirmado. Um grupo de consumidores, configurado em OutboxStreamConsumerConfig, lê a stream, e o UserRegisteredEventListener processa e confirma cada evento com ACK. O OutboxDeadLetterReclaimer reprocessa as mensagens que ficaram pendentes.

O Redis também aparece em api-messenger, que guarda a presença dos usuários em tempo real para o chat, e em api-core, que mantém o cache do catálogo de fornecedores.

Link de exemplo, OutboxStreamPublisher.kt em api-auth.

03

O grafo no Neo4j responde às consultas de recomendação

Como o projeto cumpre o requisito extra de Banco de Dados 2?

E1. Usar o Neo4j, modelando as entidades principais como grafo e respondendo a ao menos uma pergunta de negócio que exija traversal

As entidades principais do marketplace foram modeladas como grafo. Os nós incluem Supplier, SolarOffer, SolarModel, Technician, Profession, Shift e LocalUnit, ligados por relações entre ofertas, fornecedores, modelos, técnicos, profissões, turnos e experiências. As constraints e os índices do grafo ficam no schema.cypher do repositório database-bootstrap.

O grafo responde por traversal a três perguntas de negócio, sobre quais fornecedores têm ofertas válidas de um modelo, quais ofertas valem para uma unidade e quais profissionais atendem a uma necessidade. A consulta de fornecedores percorre Supplier, SolarOffer e SolarModel para ranquear os candidatos. As consultas de ofertas e de profissionais seguem o mesmo padrão.

Links de exemplo, a documentação NEO4J.md em api-recommendation e a consulta candidates.cypher em api-recommendation.

04

Oito repositórios sustentam os requisitos da disciplina

Onde está o código de cada requisito?

Repositório	Papel na disciplina	Requisitos
ai-assistant	Estado das conversas e memória de longo prazo do agente no MongoDB	M1
ai-operational	Estado das conversas do agente operacional no MongoDB	M1
api-messenger	Conversas e mensagens em coleções do MongoDB e presença em tempo real no Redis	M1, M2
api-notify	Histórico de notificações no MongoDB	M1
api-auth	Fila de eventos em Redis Stream com padrão outbox	M2
api-core	Cache do catálogo de fornecedores no Redis	M2
api-recommendation	Consultas Cypher de recomendação por traversal e documentação do grafo	E1
database-bootstrap	Nós, relações, constraints e índices do grafo	E1



Entrega acadêmica.